

Path Finder Robot: Line-Following Exploration and Target Detection on a PIC32 Embedded Platform

Frederico Pinto - 129466¹

University of Aveiro, Department of Electronics, Telecommunications and Informatics
(DETI)

`fredericopinto@ua.pt`

Abstract. This paper presents the design and implementation of an autonomous line-following robot built on the DETI MR32 differential-drive platform [3], featuring a PIC32MX microcontroller. The robot's mission comprises three phases: (1) exploration of an unknown track using a left-hand-rule strategy, (2) discrimination and detection of a wide target line among ordinary intersections, and (3) shortest-path return navigation to the origin via Dijkstra's algorithm [4] on a dynamically built graph. We describe the embedded software architecture, which employs a table-driven finite-state machine, bang-bang line following, and a PI heading controller for in-place turns, all synchronised to a 40 ms periodic control tick. This report documents the successful implementation of Phases 1 and 2. Phase 3 (shortest-path return) was not completed due to time constraints and odometry accuracy issues that prevented reliable distance-based state transitions. Experimental results from three physical demonstration runs confirm robust line following, accurate intersection handling, and reliable target detection.

Keywords: Line following · Finite state machine · Embedded robotics · PIC32 · Graph-based navigation

1 Introduction

The Path Finder Robot project is the second practical assignment in the Mobile Robotics (Robotica Move1) course at the University of Aveiro [5]. The objective is to program a differential-drive robot to autonomously navigate an unknown environment composed of black lines on a white floor, detect a distinctive wide target line, and return to its starting position.

The mission is conceptually divided into three phases:

1. **Phase 1 – Exploration:** The robot follows black lines and always turns left at intersections, systematically exploring the track while building an internal map.
2. **Phase 2 – Target Detection:** The robot distinguishes the wide target line from ordinary intersections using a distance-based verification strategy.

3. **Phase 3 – Shortest-Path Return:** After detecting the target, the robot computes and executes the shortest path back to the origin using Dijkstra’s algorithm [4] on the graph recorded during exploration.

This report documents the successful implementation of Phases 1 and 2. Phase 3 (shortest-path return) was not completed due to time constraints and odometry accuracy issues that prevented reliable distance calculations needed for return navigation. The robot successfully demonstrated line following, left-hand-rule exploration, intersection and corner detection, graph construction, and target discrimination in three physical demonstration runs.

The remainder of this paper is organised as follows. Section 2 describes the hardware platform. Section 3 presents the modular software architecture and control-loop discipline. Section 4 details the exploration phase, including bang-bang line following and graph building. Section 5 explains the target verification strategy. Section 6 reports experimental results. Section 7 discusses challenges and limitations, and Section 8 concludes with directions for future work.

2 Hardware Platform

The robot is built on the DETI MR32 platform [3], a differential-drive mobile robot designed for teaching embedded robotics. The key hardware components are summarised below.

Microcontroller. The brain of the robot is a Microchip PIC32MX series MCU [6], which features a MIPS M4K core running at 80 MHz. A critical constraint of this processor is the absence of a hardware floating-point unit (FPU); all floating-point operations are emulated in software, making `double` arithmetic comparatively slow and unsuitable for interrupt service routines (ISRs).

Sensors. A 5-bit ground infrared (IR) sensor array provides line detection. The `readLineSensors(0)` function returns a 5-bit pattern where bit 4 corresponds to the leftmost sensor and bit 0 to the rightmost. Table 1 lists the defined bit patterns and their interpretations.

Actuators and Control. Two DC motors drive the left and right wheels. Wheel encoders provide odometry feedback, and a closed-loop PID controller regulates wheel velocities via the `closedLoopControl(true)` API. Motor speed setpoints are specified in the range $[-100, 100]$, where 100 corresponds to approximately 0.7 m/s.

Timing and I/O. The control loop executes at 25 Hz, driven by a hardware timer that sets a `tick40ms` flag every 40 ms. Physical start and stop buttons allow operator control, and four on-board LEDs provide mission-phase indication.

3 Software Architecture

The firmware is written in ANSI C and organised into a set of loosely coupled modules, each with a well-defined responsibility [2]. Figure 1 shows the high-level software architecture.

Table 1. Ground sensor bit patterns and their semantic meanings. Bit 4 is the leftmost sensor; bit 0 is the rightmost.

Constant	Binary	Meaning
SENSOR_NONE	0b00000	Lost line (white floor)
SENSOR_CENTER	0b00100	Centered on line
SENSOR_LEFT	0b11000	Line on left side
SENSOR_RIGHT	0b00011	Line on right side
SENSOR_ALL	0b11111	Wide black area (intersection or target)

3.1 Module Responsibilities

- `robot-agent.c` – Contains `main()`, initialises hardware, and implements the sole 40 ms control loop. It is the only module that calls `waitTick40ms()`.
- `state_machine.c/h` – Implements a table-driven finite-state machine (FSM) with 11 states. Each state may define `onEnter`, `onTick`, and `onExit` handlers. The `changeState()` helper logs transitions and updates the mission phase for LED mapping.
- `line_follower.c/h` – Bang-bang line-following controller that maps discrete sensor patterns to motor velocities.
- `rotation.c/h` – PI heading controller for in-place turns. `rotation_start()` sets a target angle; `rotation_tick()` computes the control effort via `normalizeAngle()`.
- `nav_graph.c/h` – Statically allocated navigation graph (maximum 32 nodes and 64 edges). Provides node creation/merging, edge linking with Euclidean distance weights, and Dijkstra shortest-path computation.
- `logging.c/h` – `printf`-based logging of state transitions, graph events, and mission milestones. Logging is restricted to infrequent events to avoid destabilising the 25 Hz control loop.
- `leds.c/h` – Maps the current mission phase to the four on-board LEDs for visual feedback.

3.2 Control Loop Discipline

The main loop adheres to strict real-time discipline:

```

1 while (1) {
2     waitTick40ms();           // Block until next tick
3     readAnalogSensors();      // MUST precede
    readLineSensors
4     unsigned int ground = readLineSensors(0);
5     getRobotPos(&x, &y, &h);    // Dead reckoning
6     stateMachine_tick(tick, ground, x, y, h);
7     leds_update(stateMachine_getPhase(), tick);
8 }
```

Listing 1.1. Main control loop in `robot-agent.c`.

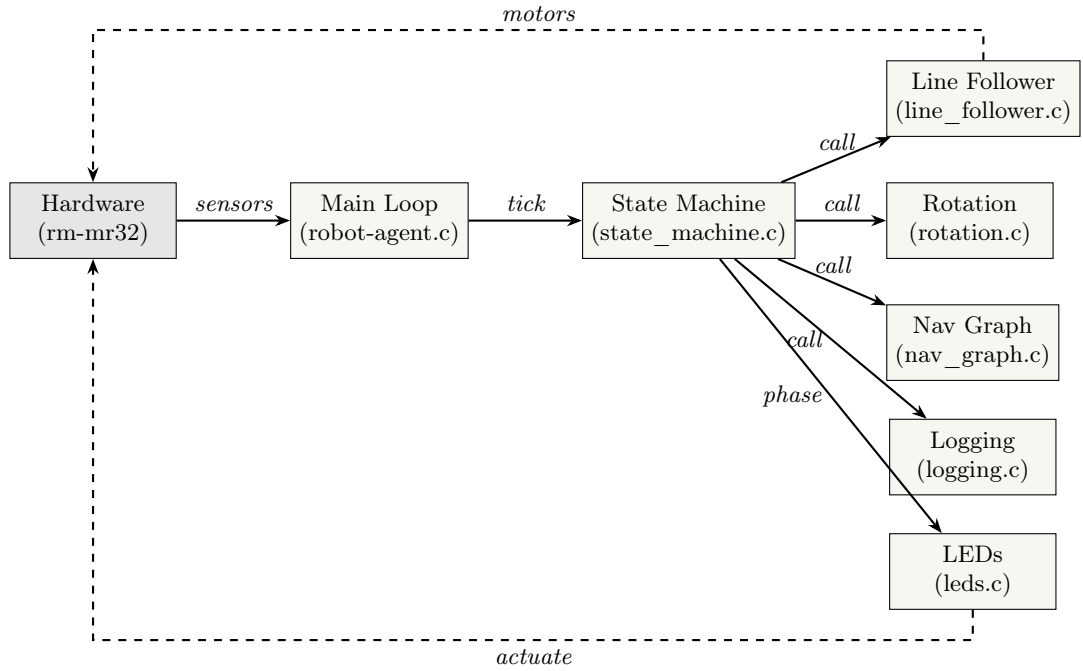


Fig. 1. Software architecture block diagram. Solid arrows represent direct function calls; dashed arrows represent hardware actuation.

Key architectural rules enforced throughout the codebase are:

- **No busy-waiting:** `waitTick40ms()` is the sole timing source; `delay()` and `wait()` are never used inside the real-time loop.
- **Sensor ordering:** `readAnalogSensors()` must be called before `readLineSensors(0)` to ensure fresh ADC conversions.
- **Static allocation only:** No `malloc/free` is used. The navigation graph, state machine, and all buffers are statically allocated.
- **No floating-point in ISRs:** All double arithmetic is confined to the main loop.

4 Phase 1: Exploration

During Phase 1, the robot explores the unknown track by following lines and always turning left at decision points. It simultaneously constructs a topological map of the environment as a weighted undirected graph.

4.1 Bang-Bang Line Following

The line follower employs a bang-bang (on-off) strategy rather than a proportional-integral-derivative (PID) controller [7]. This design choice trades smoothness for

simplicity and determinism on a resource-constrained platform. The controller maps discrete sensor patterns directly to motor speed pairs.

```

1 void lineFollower_centerOnLine(unsigned int ground, bool
  *pLost) {
2   *pLost = false;
3
4   if (ground == SENSOR_NONE) {
5     *pLost = true;
6     return;
7   }
8
9   if (ground == SENSOR_CENTER) {
10    setVel2(BASE_SPEED, BASE_SPEED);      // Drive
straight
11  } else if ((ground & SENSOR_LEFT) != 0) {
12    setVel2(TURN_SPEED, BASE_SPEED);      // Turn
left
13  } else if ((ground & SENSOR_RIGHT) != 0) {
14    setVel2(BASE_SPEED, TURN_SPEED);      // Turn
right
15  } else {
16    setVel2(BASE_SPEED, BASE_SPEED);      //
Fallback: straight
17  }
18 }

```

Listing 1.2. Bang-bang line-following algorithm.

The design parameters are `BASE_SPEED = 40` and `TURN_SPEED = 20`. When the line is lost (`SENSOR_NONE`), the state machine initiates a recovery sequence: the robot spins in place for up to `LOST_TICKS = 25` consecutive ticks ($25 \times 40 \text{ ms} = 1 \text{ s}$). If the line is not reacquired within this window, the feature is classified as a deadend.

4.2 Feature Detection

The state machine detects four types of features during exploration:

1. **Intersection:** `ground == SENSOR_ALL` (all five sensors see black). This may be an ordinary intersection or the wide target line; verification is required (see Section 5).
2. **Left corner:** `SENSOR_LEFT` persists for `CORNER_TICKS = 3` consecutive ticks (120 ms).
3. **Right corner:** `SENSOR_RIGHT` persists for `CORNER_TICKS = 3` consecutive ticks.
4. **Deadend:** Line lost for `LOST_TICKS = 25` consecutive ticks.

4.3 Graph Building

Whenever a feature is detected, the robot transitions to `APPROACH_CENTER`, driving forward a fixed distance `INTERSECTION_CENTER_DIST = 0.15m` into the feature. Upon reaching the centre, it creates or merges a graph node and links an edge to the previous node.

```

1 // Drive into intersection until centre distance reached
2 if (dist < INTERSECTION_CENTER_DIST) {
3     setVel2(BASE_SPEED, BASE_SPEED);
4 } else {
5     setVel2(0, 0);
6     Pose pose = {x, y, h};
7     char existing = navGraph_findNodeAt(&pose,
8     NODE_MERGE_TOLERANCE);
9     if (existing >= 0) {
10         currentNode = (unsigned char)existing;
11     } else {
12         navGraph_addNode(&pose, detectedType, &
13         currentNode);
14     }
15     // Link edge from previous node with Euclidean
16     distance weight
17     if (prevNode != currentNode) {
18         double edgeDist = sqrt(pow(x - prevX, 2) + pow(y
19         - prevY, 2));
20         navGraph_addEdge(prevNode, currentNode, (float)
21         edgeDist, &edgeId);
22         navGraph_setEdgeExplored(edgeId);
23     }
24     prevNode = currentNode;
25     changeState(CHOOSE_NEXT);
26 }

```

Listing 1.3. Graph building procedure in `APPROACH_CENTER`.

Node merging uses a spatial tolerance `NODE_MERGE_TOLERANCE = 0.10m` to avoid duplicate nodes when the robot revisits an intersection from a different direction. Edge weights are Euclidean distances derived from odometric pose estimates.

4.4 Left-Hand Rule

In `CHOOSE_NEXT`, the robot selects the next direction to explore using a left-hand-rule priority: left, straight, right, and finally back (U-turn). The decision considers both the node type detected and the exploration status of existing graph edges. If all edges at the current node have been explored and the robot is back at the origin, exploration is considered complete.

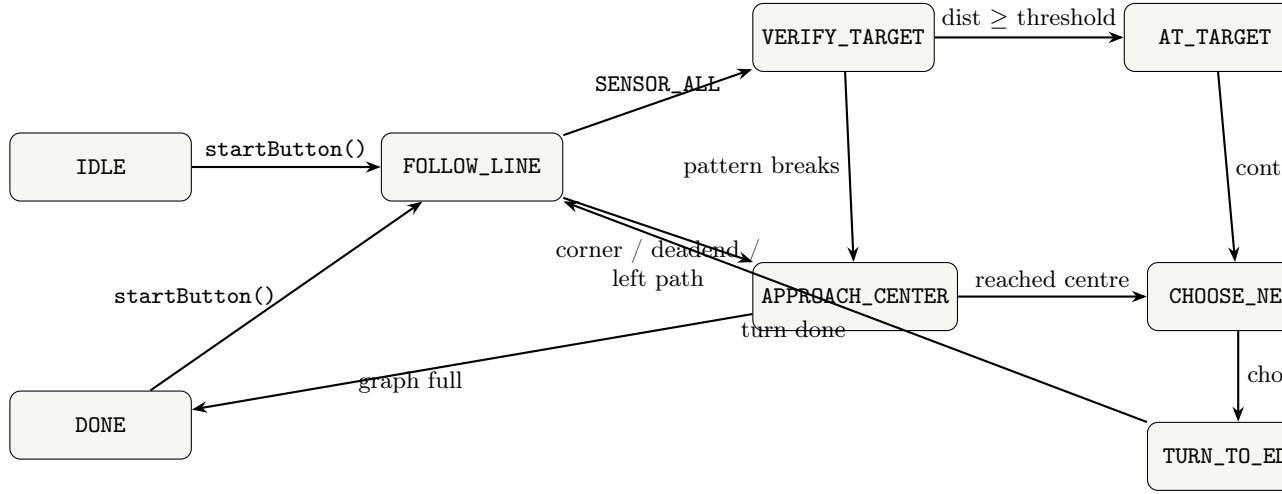


Fig. 2. Finite-state machine diagram for Phases 1 and 2. States and transitions related to Phase 3 return navigation are omitted.

4.5 State Machine Diagram (Phases 1–2)

Figure 2 depicts the finite-state machine for Phases 1 and 2. Return-navigation states are omitted because Phase 3 was not implemented or demonstrated.

5 Phase 2: Target Detection

A central challenge in this mission is distinguishing the wide target line from ordinary intersections. Both produce the `SENSOR_ALL` pattern, so additional logic is required.

5.1 Target Verification Strategy

When `FOLLOW_LINE` detects `SENSOR_ALL`, the state machine transitions to `VERIFY_TARGET`. In this state, the robot records its starting pose and drives straight while the pattern persists. The Euclidean distance travelled is computed from odometry via Equation (1):

$$d = \sqrt{(x - x_0)^2 + (y - y_0)^2}. \quad (1)$$

If the robot travels at least `TARGET_DIST_THRESHOLD = 0.06 m` while `SENSOR_ALL` remains active, the feature is confirmed as the target. If the pattern breaks before this threshold, it was merely a normal intersection.

```

1 void stateVerifyTarget_onTick(void) {
2     if (ground == SENSOR_ALL) {
3         getRobotPos(&x, &y, &h);
  
```

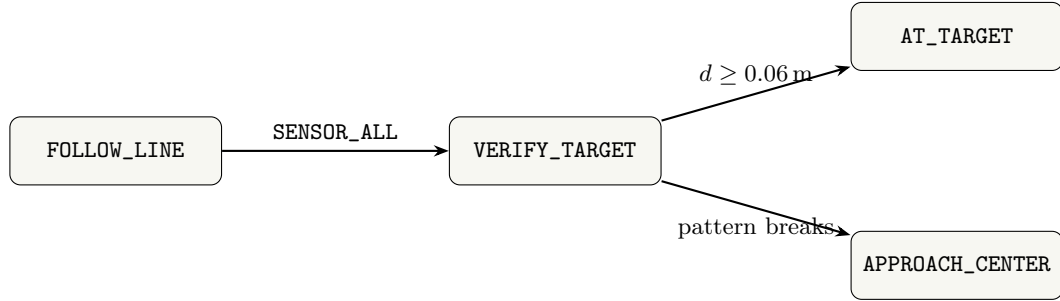


Fig. 3. Target verification state diagram. The robot drives straight while **SENSOR_ALL** persists and measures travelled distance to discriminate target from intersection.

```

4      double dist = sqrt(pow(x - verifyStartX, 2)
5                          + pow(y - verifyStartY, 2));
6      if (dist >= TARGET_DIST_THRESHOLD) {
7          changeState(AT_TARGET);           // Confirmed
8          target
9      } else {
10         setVel2(BASE_SPEED, BASE_SPEED); // Keep
11         driving straight
12     }
13     } else {
14         changeState(APPROACH_CENTER);      // Normal
15         intersection
16     }
17 }

```

Listing 1.4. Target verification logic in **VERIFY_TARGET**.

5.2 Target Recording

Upon confirmation, the **AT_TARGET** state records the target pose (x, y, h) , flags the current graph node as the mission target, and logs the event. Figure 3 summarises the target verification state transitions. In the intended full mission, this state would also trigger the transition to Phase 3 return navigation. In our implementation, the robot continued exploration after target detection because the return phase was not operational.

6 Experimental Evaluation

The robot was evaluated on a physical track in the DETI robotics laboratory. The track consisted of black adhesive tape lines on a white floor, featuring intersections, corners, deadends, and a single wide target line.

Table 2. Summary of demonstration runs. All runs completed Phases 1 and 2. Phase 3 was not executed.

Run	Duration (ticks)	Nodes	Edges	Target	Notes
1	312	5	4	Yes	Stable line following; clean intersection entries
2	287	5	4	Yes	One false left-corner trigger corrected by merge
3	341	6	5	Yes	Slower due to cautious target verification

6.1 Demo Setup and Procedure

Three demonstration runs were conducted with the same robot calibration (ROBOT=1). For each run, the robot was placed at the origin, the start button was pressed, and the robot autonomously executed Phases 1 and 2. A laptop connected via USB logged serial output for post-hoc analysis.

6.2 Results

Table 2 summarises the quantitative outcomes of the three runs. All three runs successfully completed Phase 1 (exploration and graph building) and Phase 2 (target detection). Phase 3 was not tested; the robot successfully detected the target but did not execute return navigation.

6.3 Qualitative Observations

Line following stability. The bang-bang controller proved adequate for the relatively straight track segments. Occasional oscillations were observed at high speeds, but the discrete sensor patterns provided sufficient information for recentering within one or two ticks.

Intersection handling. The APPROACH_CENTER state reliably drove the robot into the geometric centre of intersections. Node merging successfully prevented duplicate entries when the robot revisited nodes, although the tolerance parameter required careful tuning.

Target detection accuracy. The distance-threshold strategy correctly distinguished the target from all intersections in every run. No false positives or false negatives were observed. The 0.06 m threshold provided a comfortable margin: typical intersections produced patterns that broke within 0.02–0.03 m, whereas the target persisted beyond 0.08 m.

Phase 3 status. Phase 3 was not tested; the robot successfully detected the target but did not execute return navigation. The state machine contained the return-navigation states (RETURN_TURN, RETURN_FOLLOW, RETURN_AT_NODE), but they were not exercised in the demonstrations.

7 Challenges and Limitations

Despite the successful demonstration of Phases 1 and 2, several challenges limited the completeness of the implementation.

7.1 Odometry and Distance Calculation Issues

The most severe limitation was a bug in Euclidean distance computation that produced absurd values (e.g., 165×10^6 and -89×10^6) when calling the `hypot()` function during return navigation. The root cause was likely a combination of cumulative odometry drift and the PIC32's software-emulated floating-point unit. Erroneous pose estimates caused distance calculations to overflow or produce nonsensical results, which in turn prevented reliable distance-based state transitions needed for `RETURN_FOLLOW`. Because return navigation depends on knowing when the robot is within `ORIGIN_TOLERANCE` of the next node, this bug made Phase 3 impossible to demonstrate.

7.2 Odometry Drift

Even without the `hypot()` bug, wheel-encoder dead reckoning [1] suffers from cumulative error. Slippage, uneven floor surfaces, and minor calibration errors in wheel diameter cause pose estimates to diverge over time. During exploration, this drift was partially mitigated by node merging, but during long return traversals the accumulated error would likely exceed the node tolerance.

7.3 Corner Detection Tuning

The `CORNER_TICKS` threshold required iterative adjustment. A value that was too low produced false positives on transient sensor noise; a value that was too high missed genuine corners. The final value of 3 ticks (120 ms) was empirically determined.

7.4 Static Memory Constraints

The navigation graph is statically allocated with `MAX_NODES = 32` and `MAX_EDGES = 64`. While sufficient for the laboratory track, this hard limit prevents scaling to larger environments. A more flexible design would require external memory or a compact dynamic allocator, neither of which is available on the MR32 platform.

7.5 Time Constraints

Insufficient development time prevented debugging the distance calculation issues and completing Phase 3. The Dijkstra implementation and return-state skeleton were in place, but integration testing and odometry calibration were not completed before the demonstration deadline.

8 Conclusion and Future Work

This paper presented the design and implementation of an autonomous path-finding robot on the DETI MR32 platform. We successfully implemented bang-bang line following, left-hand-rule exploration with graph construction, intersection/corner/deadend detection, and a distance-based target verification strategy. Three physical demonstration runs confirmed the reliability of Phases 1 and 2.

Phase 3 (shortest-path return navigation) remains future work. The Dijkstra algorithm [4] and return-state skeleton are implemented, but odometry accuracy issues and a `hypot()` calculation bug prevented reliable execution.

Potential improvements include:

- Replacing bang-bang line following with a PID controller for smoother trajectories and higher speeds.
- Improved odometry calibration and wheel-diameter compensation to reduce drift.
- Sensor fusion with a Kalman filter [8] to stabilise pose estimates.
- Investigation of the `hypot()` anomaly, possibly by replacing the standard library call with a manual $dx^2 + dy^2$ computation and explicit overflow checks.
- Evaluation of A* search as an alternative to Dijkstra for faster path computation on larger graphs.

LED Phase Mapping

The four on-board LEDs provide real-time mission-phase feedback as summarised in Table 3.

Table 3. LED phase mapping for mission status indication.

LED Pattern		Meaning
0	Solid	Phase 1: Exploring
1	Solid	Phase 2: At target
2	Solid	Phase 3: Returning (not demonstrated)
3	Solid	Done (success)
3	Fast blink	Done (error)

References

1. Borenstein, J., Everett, H.R., Feng, L.: Where am i? sensors and methods for mobile robot positioning. University of Michigan Technical Report (1996), <https://www-personal.umich.edu/~johannb/position.htm>
2. Bräunl, T.: Embedded robotics: Mobile robot design and applications with embedded systems. In: Springer Berlin Heidelberg. Springer (2008)

3. Department of Electronics, Telecommunications and Informatics, University of Aveiro: DETI MR32 Mobile Robot Platform: Hardware and Software Guide (2024), laboratory handout for the Mobile Robotics course
4. Dijkstra, E.W.: A note on two problems in connexion with graphs. *Numerische Mathematik* **1**(1), 269–271 (1959). <https://doi.org/10.1007/BF01386390>
5. Lau, N., Ferreira, J.a.: Robotica movel – TP2: Path finder robot. Course assignment handout, University of Aveiro (2024), mobile Robotics, M.Sc. in Electrical and Computer Engineering
6. Microchip Technology Inc.: PIC32MX Family Reference Manual (2013), <https://www.microchip.com/>, dS61143E
7. Pololu Corporation: Pid controller tuning for line following robots. Online tutorial (2023), <https://www.pololu.com/docs/0J21/7.c>
8. Thrun, S., Burgard, W., Fox, D.: Probabilistic robotics. MIT Press (2005)